C for C++ Programmers — Exercise Answers

1. Introduction

1. Think about it: C uses format specifiers in `printf` while C++ uses `operator<<` or `std::format`. What advantage does the format string approach give you when writing output to a log file? What is a disadvantage?

Answer: The format string approach lets you define the entire output layout in a single, readable string. This is especially useful for log files where you want a consistent format — you can see the pattern at a glance:

```
fprintf(log, "[%04d-%02d-%02d %02d:%02d:%02d] %s: %s\n",
        year, month, day, hour, min, sec, level, msg);
```

The format string can also be stored in a variable or configuration, making it easy to change the log format without restructuring the code.

The disadvantage is that `printf` is not type-safe. If you write `%d` but pass a `double`, the compiler may warn you, but the behavior is undefined. C++ `operator<<` and `std::format` catch type mismatches at compile time.

---

2. What does this print?

```
printf("%05d %x\n", 42, 255);
```

Answer:

```
00042 ff
```

`%05d` prints `42` in a 5-character-wide field padded with leading zeros. `%x` prints `255` as lowercase hexadecimal `ff`.

---

3. Calculation: How many bytes does the string literal `"hello"` occupy in memory?

Answer: 6 bytes. The five visible characters `h`, `e`, `l`, `l`, `o` plus the null terminator `'\0'`.

---

4. Where is the bug?

```
double pi = 3.14159;
printf("Pi is %d\n", pi);
```

Answer: The format specifier `%d` expects an `int`, but `pi` is a `double`. This is undefined behavior and will print garbage. The correct specifier is `%f`:

```
printf("Pi is %f\n", pi);
```

---

5. Write a program that prints a 5x5 multiplication table using `printf` with width formatting so the columns are aligned.

Answer:

```
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

2. Pointers

1. Think about it: In C++, you can pass by reference to modify a caller's variable. Why do you think C was designed with only pass by value? What does this simplify in the language?

Answer: Pass by value only simplifies the language and its calling conventions. Every function call copies its arguments, so you always know that a function cannot modify your local variables unless you explicitly give it a pointer. This makes reasoning about code simpler: if you see `f(x)`, you know `x` cannot change (unless `x` is already a pointer). It also simplifies the compiler — there is no need for reference semantics, no hidden indirection, and no ambiguity about whether

an argument is an alias for another variable. When you do want a function to modify a variable, you explicitly pass `&x`, making the intent clear at the call site.

---

2. What does this print?

```
int a[] = {10, 20, 30, 40, 50};
int *p = a + 2;
printf("%d %d %d\n", *p, *(p - 1), p[1]);
```

Answer:

```
30 20 40
```

`p` points to `a[2]` (value 30). `*(p - 1)` is `a[1]` (value 20). `p[1]` is `*(p + 1)`, which is `a[3]` (value 40).

---

3. Calculation: On a 64-bit system, what is `sizeof(int *)`, `sizeof(char *)`, and `sizeof(double *)`?

Answer: All three are **8 bytes**. On a 64-bit system, all pointers are 8 bytes (64 bits) regardless of the type they point to. The type only affects pointer arithmetic and dereferencing, not the size of the pointer itself.

---

4. Where is the bug?

```
int *get_value(void) {
    int result = 42;
    return &result;
}
```

Answer: The function returns a pointer to a local variable. When `get_value` returns, `result` is destroyed (its stack frame is reclaimed), and the returned pointer becomes a **dangling pointer**. Dereferencing it is undefined behavior. To fix this, either return the value directly (`return result` with return type `int`), use a `static` local, or allocate memory with `malloc`.

---

5. What does this print?

```
int x = 10;
int *p = &x;
int **pp = &p;
**pp = 20;
printf("%d\n", x);
```

Answer:

```
20
```

`pp` points to `p`, and `p` points to `x`. `**pp` dereferences `pp` to get `p`, then dereferences `p` to get `x`. Assigning `**pp = 20` changes `x` to 20.

---

6. Where is the bug?

```
struct song {
    char title[40];
    int year;
};

struct song *p = NULL;
printf("%s\n", p->title);
```

Answer: `p` is `NULL`, so `p->title` dereferences a null pointer. This is undefined behavior and will typically crash with a segmentation fault. You must point `p` to a valid `struct song` before accessing its fields:

```
struct song track = {"Something", 1985};
struct song *p = &track;
printf("%s\n", p->title);
```

---

7. Write a program that declares an array of 5 integers, uses a pointer to iterate through the array, and prints each element along with its memory address.

Answer:

```
#include <stdio.h>

int main(void) {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;

    for (int i = 0; i < 5; i++) {
        printf("arr[%d] = %d at address %p\n", i, *(p + i), (void *)(p + i));
    }
    return 0;
}
```

Output (addresses will vary):

```
arr[0] = 10 at address 0x7ffd...
arr[1] = 20 at address 0x7ffd...
arr[2] = 30 at address 0x7ffd...
arr[3] = 40 at address 0x7ffd...
arr[4] = 50 at address 0x7ffd...
```

3. Allocating Memory

1. Think about it: Why would you choose `calloc` over `malloc` followed by `memset` to zero?

Answer: `calloc` combines allocation and zeroing in a single call, which is simpler and less error-prone. It also takes the element count and size as separate arguments (`calloc(n, sizeof(int))`), which protects against integer overflow — `calloc` checks whether `n * size` overflows before allocating, while `malloc(n * sizeof(int))` silently overflows if `n` is very large. Using `calloc` is also clearer about intent: "I want `n` zero-initialized elements."

---

2. What does this print?

```c
#include <stdio.h>

void counter(void) {
    static int n = 0;
    n++;
    printf("%d ", n);
}

int main(void) {
    counter(); counter(); counter();
    return 0;
}
```

Answer:

```
1 2 3
```

The `static` keyword means `n` is initialized once and retains its value between calls. The first call increments `n` from 0 to 1, the second from 1 to 2, and the third from 2 to 3.

---

3. Calculation: On a system where `int` is 32 bits, how many bytes does `malloc(5 * sizeof(int))` allocate?

Answer: 20 bytes. An `int` that is 32 bits is 4 bytes, and `5 * 4 = 20`.

---

4. Where is the bug?

```c
int *p = malloc(10 * sizeof(int));
for (int i = 0; i < 10; i++) {
    p[i] = i;
}
```

```c
free(p);
printf("%d\n", p[0]);
```

Answer: The code accesses `p[0]` after calling `free(p)`. This is a **use-after-free** bug — undefined behavior. Once memory is freed, the pointer becomes invalid and must not be dereferenced. The `printf` must be moved before the `free`, or the code should be restructured so the data is no longer needed after freeing.

---

5. Where is the bug?

```c
int *a = malloc(5 * sizeof(int));
int *b = a;
free(a);
free(b);
```

Answer: This is a **double free**. Both `a` and `b` point to the same allocated memory. After `free(a)`, that memory is released. Calling `free(b)` frees the same block a second time, which is undefined behavior and can corrupt the heap. The fix is to only free the memory once. If you need to track that it has been freed, set the pointer to `NULL` after freeing:

```c
free(a);
a = NULL;
b = NULL;
```

---

6. Write a program that uses `malloc` to allocate an array of `n` integers (where `n` is provided by the user via `scanf`), fills the array with squares (0, 1, 4, 9, . . .), prints them, and frees the memory.

Answer:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);

    int *arr = malloc(n * sizeof(int));
    if (arr == NULL) {
        fprintf(stderr, "Allocation failed\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        arr[i] = i * i;
```

```
    }

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);
    return 0;
}
```

Example output for `n = 5`:

```
0 1 4 9 16
```

4. Strings

1. Think about it: Why does `strcmp` return 0 for equal strings rather than 1? How does this relate to the function's actual purpose?

Answer: `strcmp` computes the difference between two strings, not a boolean "are they equal?" check. It returns a negative value if the first string comes before the second lexicographically, a positive value if it comes after, and zero if there is no difference. Zero means "no difference," which is the natural result for equal strings. This three-way return value is useful for sorting (e.g., with `qsort`). If it returned 1 for equal, you would lose the ability to determine ordering in a single call.

---

2. What does this print?

```
char s[] = "Ghostbusters";
printf("%zu %zu\n", strlen(s), sizeof(s));
```

Answer:

```
12 13
```

`strlen(s)` returns 12 — the number of characters before the null terminator. `sizeof(s)` returns 13 — the total size of the array in bytes, including the null terminator.

---

3. Calculation: What is `sizeof(buf)` for `char buf[20] = "Hola";`?

Answer: 20. `sizeof` returns the size of the declared array, not the length of the string stored in it. The array is declared as 20 bytes, so `sizeof(buf)` is 20 regardless of the string's content.

---

4. Where is the bug?

```
char buf[10] = "Livin'";
strcat(buf, " on a Prayer");
printf("%s\n", buf);
```

Answer: Buffer overflow. `buf` is 10 bytes. `"Livin'"` uses 7 bytes (6 characters + `'\0'`), leaving 3 bytes free. `" on a Prayer"` is 12 characters, so the combined string needs 19 bytes (6 + 12 + `'\0'`), which far exceeds the 10-byte buffer. `strcat` writes past the end of `buf`, causing undefined behavior.

---

5. Where is the bug?

```
char *a = "Hello";
char *b = "Hello";
if (a == b) {
    printf("Equal\n");
} else {
    printf("Not equal\n");
}
```

Answer: The `==` operator compares the **addresses** of the two string literals, not their contents. The output is **not guaranteed**. Some compilers merge identical string literals (so `a == b` would be true), but others store them separately (so `a == b` would be false). To correctly compare string content, use `strcmp`:

```
if (strcmp(a, b) == 0) {
    printf("Equal\n");
}
```

---

6. Write a program that reads a string from the user, reverses it in place using pointer arithmetic, and prints the result.

Answer:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf[100];
    printf("Enter a string: ");
    if (fgets(buf, sizeof(buf), stdin) == NULL) {
        return 1;
    }

    /* Remove trailing newline from fgets */
```

```c
    size_t len = strlen(buf);
    if (len > 0 && buf[len - 1] == '\n') {
        buf[len - 1] = '\0';
        len--;
    }

    /* Reverse in place using two pointers */
    char *left = buf;
    char *right = buf + len - 1;
    while (left < right) {
        char tmp = *left;
        *left = *right;
        *right = tmp;
        left++;
        right--;
    }

    printf("Reversed: %s\n", buf);
    return 0;
}
```

Example:

```
Enter a string: Hola
Reversed: aloH
```

5. Standard I/O

1. Think about it: Why does `scanf` need the `&` operator for scalar variables but not for arrays?

Answer: `scanf` needs a pointer to the location where it should store the value it reads. For a scalar variable like `int x`, `x` is the value, not an address, so you must pass `&x` to give `scanf` the address. For an array like `char name[50]`, the array name already decays to a pointer to its first element. Writing `name` in an expression is equivalent to `&name[0]`, so no explicit `&` is needed.

---

2. What does this print?

```c
char buf[50];
sprintf(buf, "%s: %d", "Score", 100);
printf("%zu\n", strlen(buf));
```

Answer:

```
10
```

`sprintf` writes `"Score: 100"` into `buf`. That string is 10 characters: "Score" is 5, ":" is 2, "100" is 3, totaling 10. `strlen` returns 10.

---

3. Calculation: If `buf` is declared as `char buf[20]` and you call `snprintf(buf, sizeof(buf), "Year: %d", 1984)`, how many characters (excluding the null terminator) are written to `buf`?

Answer: 10. The formatted string is `"Year: 1984"` which is 10 characters. Since this fits within the 20-byte buffer, all 10 characters are written (plus a null terminator).

---

4. Where is the bug?

```
int x;
scanf("%d", x);
```

Answer: Missing the `&` operator. `scanf` needs the **address** of `x` to store the value, but this passes the **value** of `x` (which is uninitialized). This is undefined behavior and will likely crash. The fix:

```
scanf("%d", &x);
```

---

5. Where is the bug?

```
FILE *f = fopen("noexist.txt", "r");
fprintf(f, "Hello\n");
fclose(f);
```

Answer: The code does not check whether `fopen` succeeded. If the file does not exist, `fopen` returns `NULL`, and `fprintf(NULL, ...)` is undefined behavior (typically a crash). The fix:

```
FILE *f = fopen("noexist.txt", "r");
if (f == NULL) {
    perror("fopen");
    return 1;
}
fprintf(f, "Hello\n");
fclose(f);
```

Also, opening a nonexistent file with `"r"` (read) will always fail. If the intent is to write, the mode should be `"w"`.

---

6. Think about it: You run `./program > output.txt` and your program contains both `printf` and `fprintf(stderr, ...)` calls. Which messages appear in `output.txt` and which appear on the screen? Why?

Answer: The `>` operator redirects `stdout` to `output.txt`, so all `printf` output (which goes to `stdout`) ends up in the file. `stderr` is **not** redirected by `>`, so `fprintf(stderr, ...)` messages still appear on the screen. This is by design — error messages should remain visible even when normal output is redirected. To redirect `stderr` as well, you would use `2> errors.txt` or `2>&1` to merge it with `stdout`.

---

7. Write a program that opens a text file, writes five lines to it, closes it, reopens it for reading, reads and prints each line using `fgets`, then closes it again.

Answer:

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("test_output.txt", "w");
    if (f == NULL) {
        perror("fopen write");
        return 1;
    }

    fprintf(f, "Line 1: Hola\n");
    fprintf(f, "Line 2: Que tal\n");
    fprintf(f, "Line 3: Muy bien\n");
    fprintf(f, "Line 4: Gracias\n");
    fprintf(f, "Line 5: Adios\n");
    fclose(f);

    f = fopen("test_output.txt", "r");
    if (f == NULL) {
        perror("fopen read");
        return 1;
    }

    char line[100];
    while (fgets(line, sizeof(line), f) != NULL) {
        printf("%s", line);
    }
    fclose(f);

    return 0;
}
```

Output:

```
Line 1: Hola
```

```
Line 2: Que tal
Line 3: Muy bien
Line 4: Gracias
Line 5: Adios
```

6. Low-Level I/O

1. Think about it: Why would you use low-level `read`/`write` instead of `fprintf`/`fscanf`? When would `stdio` be the better choice?

Answer: Low-level I/O is useful when you need precise control: writing binary data without translation, avoiding buffering overhead, working with file descriptors from system calls (e.g., `pipe`, `socket`), or performing atomic operations with `pread`/`pwrite`. It is also necessary when interacting with devices or special files.

`stdio` is the better choice for formatted text I/O — it provides convenient formatting with `printf`/`scanf`, handles buffering automatically for better performance on many small writes, and is portable across all platforms. For most programs that read and write text files, `stdio` is simpler and sufficient.

---

2. What does this print?

```
write(1, "ABC", 3);
write(1, "DEF\n", 4);
```

Answer:

```
ABCDEF
```

The first `write` outputs `ABC` (no newline). The second outputs `DEF\n`. Since `write` does no buffering, the bytes appear immediately and consecutively, producing `ABCDEF` followed by a newline.

---

3. Calculation: If `read(fd, buf, 1024)` returns 512, what does that tell you? Does it mean there was an error?

Answer: No, it is not an error. A return value of 512 means that 512 bytes were successfully read. `read` can return fewer bytes than requested for several reasons: the file may have only 512 bytes remaining, the data is coming from a pipe or socket and only 512 bytes are available right now, or the operating system interrupted the read. A return value of 0 means end of file, and -1 means an error occurred. Any positive value is a successful read.

---

4. Where is the bug?

```
int fd = open("newfile.txt", O_WRONLY | O_CREAT);
write(fd, "Hello\n", 6);
close(fd);
```

Answer: When `O_CREAT` is used, `open` requires a third argument specifying the file permissions. Without it, the permissions are undefined (whatever garbage value happens to be on the stack). The fix:

```
int fd = open("newfile.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

---

5. Think about it: Explain the difference between `lseek(fd, 0, SEEK_END)` and `lseek(fd, -1, SEEK_END)`. What does each return?

Answer: `lseek(fd, 0, SEEK_END)` moves the file position to exactly the end of the file — one byte past the last byte. It returns the file size in bytes. For a 100-byte file, it returns 100 and positions at byte 100 (where a subsequent `read` would return 0 since there is no data past the end).

`lseek(fd, -1, SEEK_END)` moves to one byte *before* the end — the position of the last byte in the file. It returns the file size minus 1. For a 100-byte file, it returns 99 and positions at byte 99 (the last byte). A subsequent `read` would read that one last byte.

---

6. Write a program that uses low-level I/O (`open`, `read`, `write`, `close`) to copy the contents of one file to another. The source and destination filenames should be taken from `argv`.

Answer:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        write(2, "Usage: copy src dst\n", 20);
        return 1;
    }

    int src = open(argv[1], O_RDONLY);
    if (src == -1) {
        perror("open source");
        return 1;
    }

    int dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst == -1) {
```

```
        perror("open dest");
        close(src);
        return 1;
    }

    char buf[4096];
    ssize_t n;
    while ((n = read(src, buf, sizeof(buf))) > 0) {
        ssize_t written = 0;
        while (written < n) {
            ssize_t w = write(dst, buf + written, n - written);
            if (w == -1) {
                perror("write");
                close(src);
                close(dst);
                return 1;
            }
            written += w;
        }
    }

    if (n == -1) {
        perror("read");
    }

    close(src);
    close(dst);
    return (n == -1) ? 1 : 0;
}
```

7. Odds and Ends

1. Think about it: In C++ you would use exceptions for error handling. In C there are no exceptions. What strategies can you use to handle errors in deeply nested function calls? When is `exit` appropriate and when is it not?

Answer: The main strategies are:

- **Return codes:** Each function returns an error code (0 for success, -1 or an error enum for failure). The caller checks the return value and either handles the error or propagates it up. This is the most common approach.
- **`errno` and `perror`:** Many C library functions set `errno` on failure. You can call `perror` or `strerror(errno)` to get a human-readable message.
- **`goto` cleanup:** When a function acquires multiple resources, use `goto` to jump to cleanup labels that release resources in reverse order. This is C's version of RAII.

- **`exit`:** Terminates the program immediately.

`exit` is appropriate for truly fatal errors where there is no reasonable recovery — for example, failing to open a required configuration file at startup, or running out of memory for a critical allocation. It is not appropriate in libraries (which should let the caller decide how to handle errors) or when partial cleanup is needed (since `exit` bypasses the normal unwinding of the call stack).

---

2. What happens here?

```c
#include <stdlib.h>
#include <stdio.h>

void cleanup(void) {
    printf("Adios!\n");
}

int main(void) {
    atexit(cleanup);
    printf("Starting...\n");
    exit(0);
}
```

Answer:

```
Starting...
Adios!
```

`atexit` registers `cleanup` to be called when the program exits. `printf` prints `"Starting..."`, then `exit(0)` triggers the `atexit` handlers, which calls `cleanup`, printing `"Adios!"`. The `exit` function flushes all `stdio` streams and calls `atexit` handlers before terminating.

---

3. Where is the bug?

```c
char *get_greeting(void) {
    char buf[50];
    sprintf(buf, "Hola, mundo");
    return buf;
}
```

Answer: The function returns a pointer to the local array `buf`. When `get_greeting` returns, `buf` is destroyed (its stack frame is reclaimed), and the returned pointer becomes a **dangling pointer**. Dereferencing it is undefined behavior. The fix is to use `strdup` to allocate heap memory:

```c
char *get_greeting(void) {
    char buf[50];
```

```
    sprintf(buf, "Hola, mundo");
    return strdup(buf);  /* caller must free */
}
```

Or have the caller pass in a buffer:

```
void get_greeting(char *buf, size_t size) {
    snprintf(buf, size, "Hola, mundo");
}
```

---

4. Think about it: You call a function `char *get_name(int id)` from a library. How would you determine whether you need to `free` the returned pointer?

Answer: Check the documentation. Look for phrases like "the caller must free the returned pointer" or "the returned pointer points to a static buffer." If the documentation does not say, read the source code to see if the function calls `malloc`, `calloc`, or `strdup` internally. You can also check if the function returns a pointer to a `static` buffer (which you must not free) or to a field within a library-managed struct. As a last resort, you can test: call the function twice and see if both returned pointers point to the same address (suggesting a static buffer). When in doubt, consult the library's header files for ownership annotations or comments.

---

5. Where is the bug? (Hint: ownership)

```
char *name = strdup("Walking on Sunshine");
char *alias = name;
free(name);
printf("%s\n", alias);
```

Answer: `alias` is assigned the same address as `name`. After `free(name)`, both `name` and `alias` point to freed memory. The `printf` dereferences `alias`, which is a **use-after-free** — undefined behavior. The fix is to either print before freeing, or duplicate the string for `alias`:

```
char *alias = strdup(name);
free(name);
printf("%s\n", alias);
free(alias);
```

---

6. Calculation: Given `int nums[] = {5, 10, 15, 20};`, what is the value of `sizeof(nums) / sizeof(nums[0])`?

Answer: 4. The array contains 4 `int` elements. `sizeof(nums)` is `4 * sizeof(int)` (typically 16 bytes), and `sizeof(nums[0])` is `sizeof(int)`

(typically 4 bytes). Dividing gives the number of elements: 4. This is a common C idiom for computing array length at compile time.

---

7. What does this print?

```c
int compare_desc(const void *a, const void *b) {
    int ia = *(const int *)a;
    int ib = *(const int *)b;
    return (ib > ia) - (ib < ia);
}

int main(void) {
    int vals[] = {3, 1, 4, 1, 5};
    qsort(vals, 5, sizeof(int), compare_desc);
    printf("%d %d %d %d %d\n", vals[0], vals[1], vals[2], vals[3], vals[4]);
    return 0;
}
```

Answer:

```
5 4 3 1 1
```

The comparison function compares `ib` to `ia` (instead of `ia` to `ib`), which reverses the sort order. `qsort` sorts the array in **descending** order: 5, 4, 3, 1, 1.

---

8. Write a program that uses `qsort` to sort an array of strings in reverse alphabetical order. Write a custom comparison function that calls `strcmp` with the arguments swapped.

Answer:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int compare_reverse(const void *a, const void *b) {
    const char *sa = *(const char **)a;
    const char *sb = *(const char **)b;
    return strcmp(sb, sa);  /* swapped for reverse order */
}

int main(void) {
    const char *words[] = {"delta", "alpha", "charlie", "bravo", "echo"};
    int n = sizeof(words) / sizeof(words[0]);

    qsort(words, n, sizeof(char *), compare_reverse);
```

```
    for (int i = 0; i < n; i++) {
        printf("%s\n", words[i]);
    }
    return 0;
}
```

Output:

```
echo
delta
charlie
bravo
alpha
```

---

9. Write a program in C++ that uses `extern "C"` to call the C function `strlen` from `<string.h>`, passes it a string, and prints the result. Compile it with `c++` to verify it works.

Answer:

```
#include <cstdio>

extern "C" {
    #include <string.h>
}

int main() {
    const char *msg = "Hola, amigo";
    size_t len = strlen(msg);
    printf("'%s' has %zu characters\n", msg, len);
    return 0;
}
```

Output:

```
'Hola, amigo' has 11 characters
```

Note: In practice, C++ standard headers like `<cstring>` already handle the `extern "C"` linkage for you. This exercise demonstrates the mechanism explicitly.